

Demonstrating Good Practice in Coding

Mindaugas Taujanskas

I. DON'T REPEAT YOURSELF (DRY)

DRY is a simple practice which promotes refactoring commonly used procedures into functional units which can be reused either to minimise verbatim repetition, or to parameterise a certain routine into a more generic building block. Consider an example from my removals system:

```
class ValidationMixin:
    def is_optional(self) -> bool:
        self._is_optional = getattr(
            self, "_is_optional", False
        )
        return self._is_optional

    def set_optional(self, to: bool) -> None:
        self._is_optional = to
        return self

    def set_validation_trigger(self, fn) -> None:
        self._validation_trigger = fn
        return self

    def register_validation_func(self, fn:
        ValidationFnT) -> None:
        if not hasattr(self, "state"):
            raise RuntimeError("Base class does not
                inherit StyledWidget")
        if not hasattr(self, "_validation_trigger"):
            raise RuntimeError(
                "Tried to register validation function with
                no trigger"
            )
        self._validation_fn = fn
        self._validation_trigger.connect(self.
            _validate_callback)
        return self

    def _validate_callback(self) -> None:
        data = self.serialize()
        if not data and self.is_optional():
            self.state.emit("")
            return
        self.state.emit(
            "" if self._validation_fn(data) else "error"
        )

    def is_valid(self) -> bool:
        data = self.serialize()
        if not data:
            return self.is_optional()
        if not hasattr(self, "_validation_fn"):
            return True
        return self._validation_fn(data)

    def serialize(self) -> object:
        raise NotImplementedError
```

My rationale behind creating the `ValidationMixin` class is due to earlier stages of development where I found myself recreating methods to validate form data (like emails, passwords, numbers,) and this became a very quick cause for concern when I found myself creating improved iterations of essentially the same validation routines over time, but leaving older iterations remnant, thus meaning I'd have to go back and

rewrite them. As the number of UI field components grew it created increasingly more technical debt, so I extracted the key methods and logic into a mix-in class (technically-speaking, classes that *specifically* extend the base functionality of another class), and subclassed it against every field component so that their behaviour would be guaranteed to be consistent.

A. Separation of concerns

A certain aspect where DRY may seem to be (falsely) violated, is with the model-view-controller (MVC) pattern. Consider the login procedure implemented at three stages, firstly as a SQL procedure:

```
CREATE FUNCTION login_user(p_email TEXT, p_password
    TEXT)
BEGIN
    normalized_email = normalize_email(p_email);

    SELECT user_id, password_hash, user_role
    INTO stored_user_id, stored_hash, stored_user_role
    FROM Users
    WHERE email = normalized_email;

    IF NOT FOUND THEN
        RETURN QUERY
        SELECT 'Invalid email or password', NULL, NULL;
    END IF;

    IF crypt(p_password, stored_hash) = stored_hash
    THEN
        RETURN QUERY
        SELECT
            '',
            sign(/* ... */),
            stored_user_role;
    END IF;

    RETURN QUERY
    SELECT 'Invalid email or password', NULL, NULL;
END;
```

Which validates the email parameter, verifies it exists, and then validates the password and returns a session context once everything seems OK. Then the model implementation for the database in Python:

```
def proc_login_user(email: str, password: str) ->
    list[DictRow]:
    return call_proc("login_user", (email, password))
```

This closes together the boundary between database and application layer. Finally, in the user model:

```

class User:
    def __init__(
        self,
        email: str,
        password: str,
        *,
        token_role_pair: tuple[str, str] | None = None
    ) -> None:
        if token_role_pair is not None:
            self.token, self.role = token_role_pair
            return
        error, *token_role_pair = db.proc_login_user(
            email, password)
        self.maybe_raise_exception(error)
        self.token, self.role = token_role_pair

```

Could we not have just used `call_proc` ourselves and saved the need to create a separate module? Better yet, we could've just concocted our own SQL `INSERT` statement in the `User` class, and saved ourselves from nesting two layers of interaction.

No. Each layer adds another level of flexibility and separates responsibility for: (a) who must maintain database-level security (b) how the database interface is implemented (PostgreSQL, MSSQL?) (c) what it means to login, and what the fail-states are.

DRY takes least precedence over overarching principles such as the separation of concerns, it tends to follow as a result of being disciplined to other good practices.

II. KEEP IT SIMPLE, STUPID (KISS)

KISS retains its relevancy at all stages of development. It is without a doubt that software projects will become increasingly complex as they grow, but ideally only as the sum of their parts, and not because of obscure, cryptic parts which form an ugly hornet's nest of complex interactions and couplings.

A small snippet of my project which I found a little averse to this principle is the following:

```

def get_widget(self, name: str, default: None = None)
    -> "QWidget | None":
    # NOTE: :)
    if any((which := field).name == name for field in
        self.fields):
        return which
    return default

```

The canonical form for writing this would be simply:

```

def get_widget(self, name: str, default: None = None)
    -> "QWidget | None":
    for field in self.fields:
        if field.name == name:
            return field
    return default

```

But the former read left-to-right explains succinctly the mechanism of the function, let alone the function's name itself, although it utilises a fairly cryptic feature of the Python language.

III. THE SOLID PRINCIPLES

SOLID is a very well-studied and understood set of principles that underline object-oriented design, which outlines how classes should be created, maintained, constrained, related and designed. It promotes loosely-coupled classes which

subsequently weighs positively in on their maintainability, modularity and testing.

I won't go into detail on the specific principles, but they are a quintessential baseline for developing a well-oiled object-oriented application.